

Data Storage in Big Data

Big Data Storage concept

- Big Data analytics get data from different sources. Data obtained from these different sources come with different format and structures that becomes very difficult to process these data directly.
- To overcome these incompatibilities and prepare data for storage and processing, data wrangling is necessary. Data wrangling is the process of cleaning, organizing and transform row data into structured and usable format for analysis ,
- Data wrangling includes steps to filter, cleanse and otherwise prepare the data for downstream analysis. From a storage perspective, a copy of the data is first stored in its acquired format, and, after wrangling, the prepared data needs to be stored again.
BRIEFLY EXPLAIN AT WHICH CONDITION THE DATA STORAGE IS REQUIRED
- Data storage is required whenever the following occurs:
 - External and Internal Datasets:
 - external datasets are acquired, or internal data will be used in a Big Data environment
 - Data Manipulation:
 - data is manipulated to be made cooperative and manageable for data analysis
 - ETL and Analytical Outputs
 - data is processed via an ETL activity, or output is generated as a result of an analytical operation that need to be stored.

Data Storage in Big Data

Big Data Storage concept

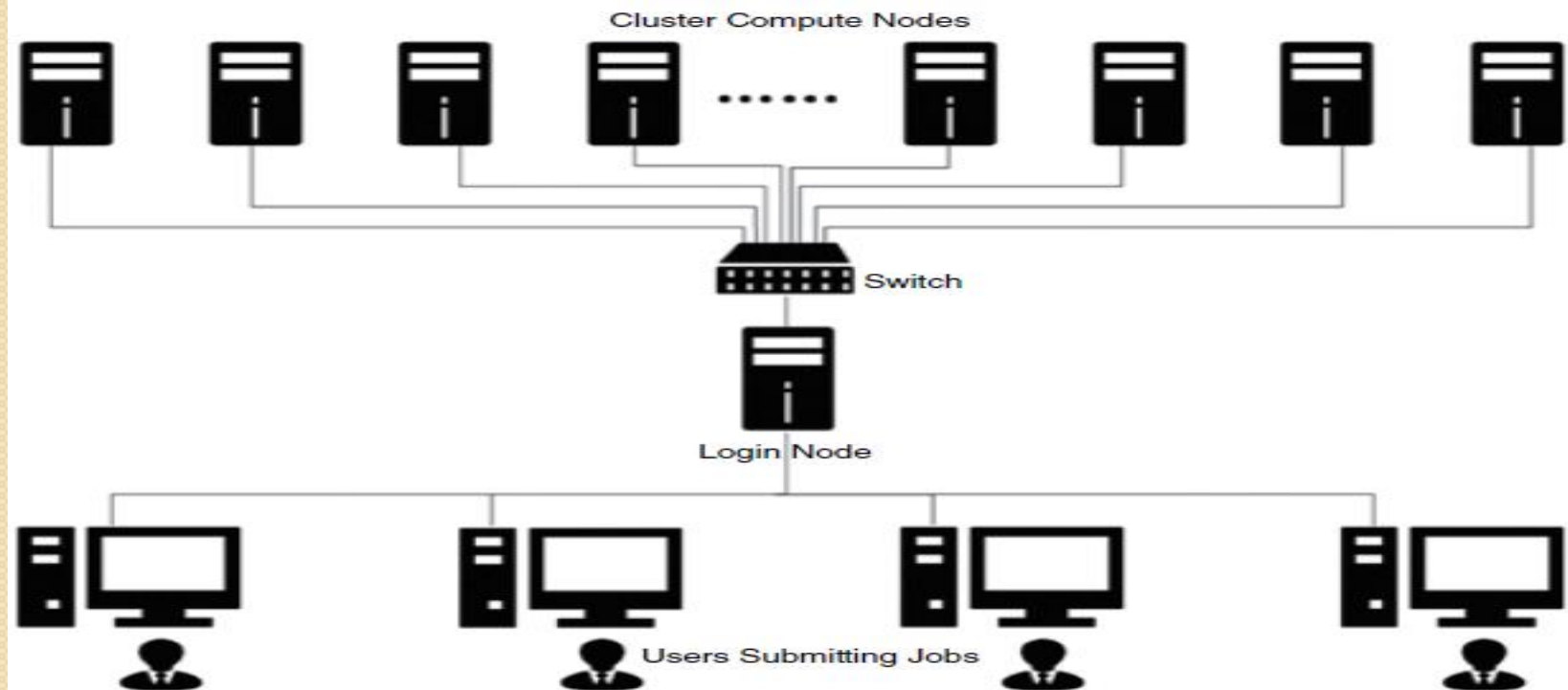
➤ Big Data storage use some of the following technologies that allow data to be stored and manipulated in efficiently and in a cost-effective ways:

- Cluster
- Sharding
- Replication
- NoSQL
- ACID

Clusters Storage

A group of interconnected servers working together as a single system to store, process, and analyze data.

- **Clusters:** It is a tightly coupled collection of servers, or nodes
- It is a distributed or parallel computing system comprising multiple stand-alone PCs connected together working as a single, integrated, highly available resource.
- A cluster can execute a task by splitting it into small pieces and distributing their execution onto different computers that belong to the cluster.
- Clusters may be configured for various purposes such as web-based services or computational-intensive workloads.



Clusters Storage

Type of Clusters: The clusters may be classified into two major types:

High availability Clusters: is a group of servers (nodes) that work together to ensure that an application or service remains operational and accessible even if one or more of the servers fail.

- These types of clusters are designed to minimize downtime and provide uninterrupted service when nodes fail.
- Nodes in a highly available cluster must have access to a shared storage. Such systems are often used for failover and backup purposes.
- Without clustering the nodes if the server running an application goes down, the application will not be available until the server is up again.

Load balancing Clusters is a group of servers (nodes) that work together to distribute incoming workloads evenly across all nodes in the cluster

- These types of clusters are **designed to distribute** workloads across different cluster nodes to share the service load among the nodes.
- If a node in a load-balancing cluster goes down, the load from that node is switched over to another node. This is achieved by having identical copies of data across all the nodes, so the remaining nodes can share the increase in load.
- The main objective of load balancing is to **optimize the use of resources, minimize response time, maximize throughput, and avoid overload on any one of the resources.**

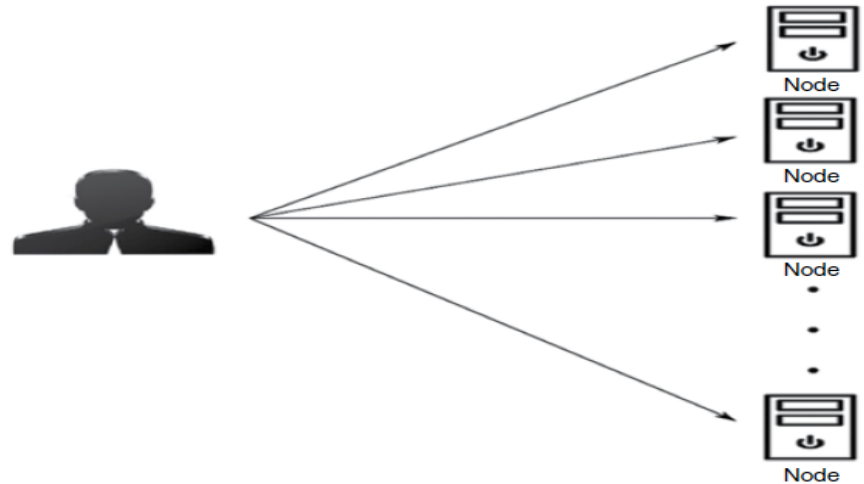
Clusters Storage

Clusters Structure:

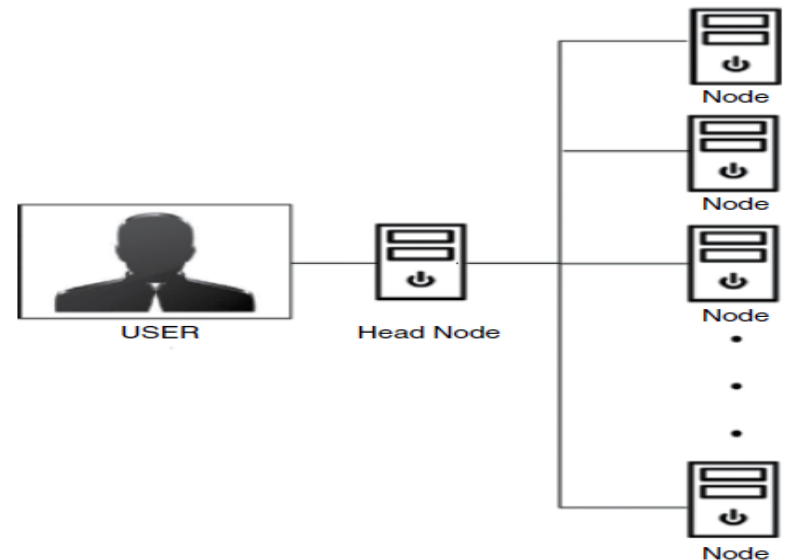
Clusters are deployed to improve performance and availability. Based on how these computers are linked together, cluster structure is classified into two types:

- **Symmetric cluster** is a type of cluster structure in which each node functions as an individual computer capable of running applications. Additional machines can be added as needed or removed from a sub-network..
- **Asymmetric clusters** are a type of cluster structure in which one machine acts as the head node, and it serves as the gateway between the user and the remaining nodes.

Symmetric cluster



Asymmetric clusters



Clusters Storage

Distribution Models for Clusters

There are several distribution models with which an increase in data volume and large volumes of read or write requests can be handled, and the network can be made highly available. The following are some common useful Distribution Models:

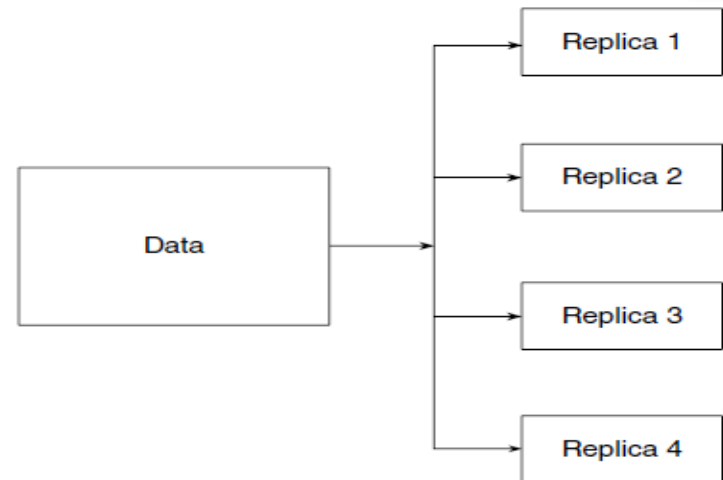
Replication: Creating copies of data across multiple nodes or servers.

- A process of creating copies of the same set of data across multiple servers/nodes. The copy of a block is called replica.

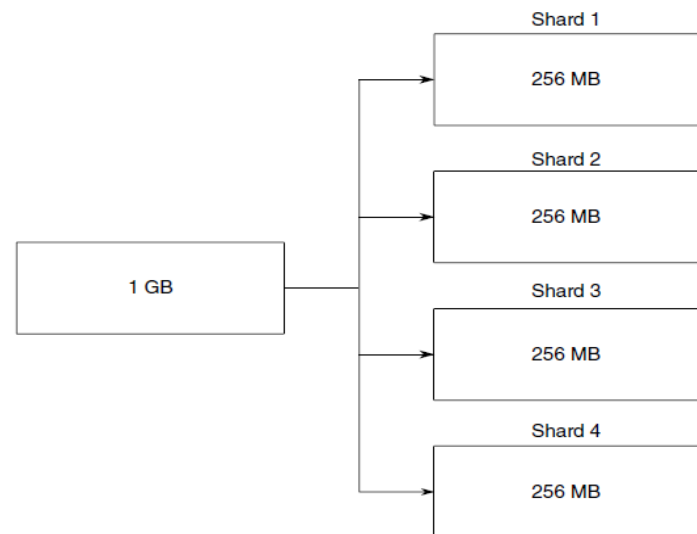
Sharding: Dividing a large dataset into smaller, manageable pieces (shards), each stored on a different node

- A process of placing different sets of data on different nodes. It is the process of partitioning very large data sets into smaller and easily manageable chunks called shards.
- The partitioned data (shards) are distributing them across multiple machines called nodes. No two shards of the same file are stored in the same node.

Replication

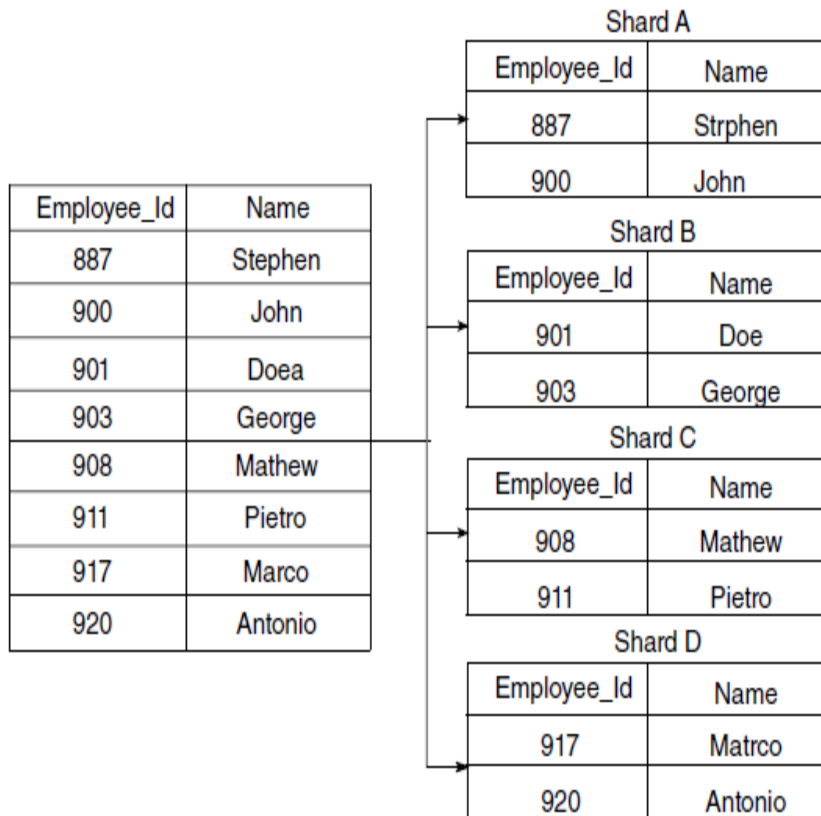


Sharding

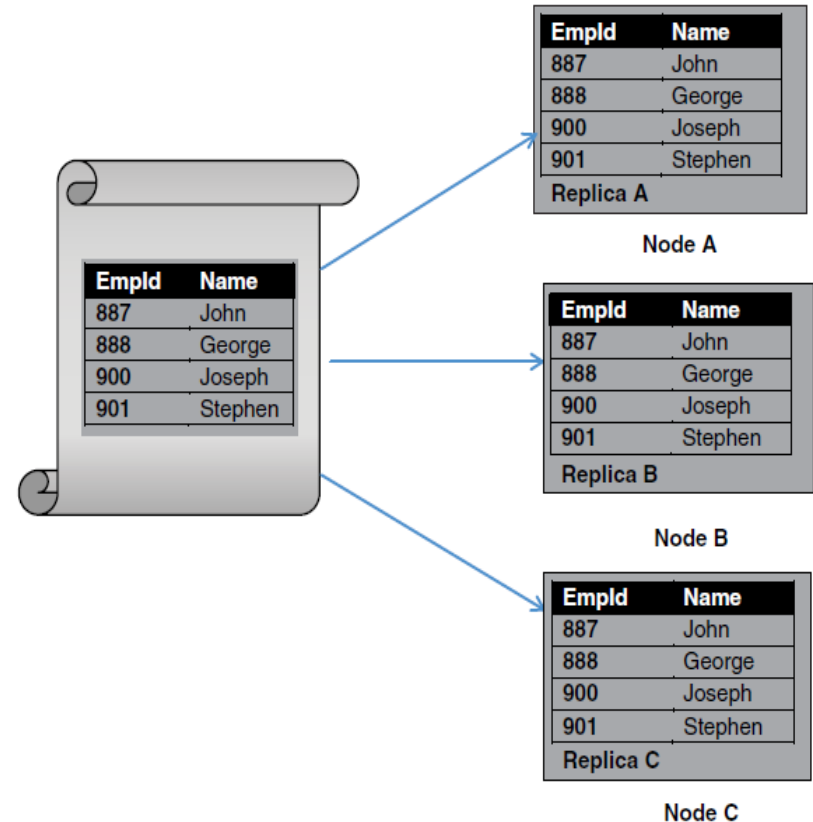


Clusters Storage

Sharding



Replication



NoSQL Database Storage

NoSQL Databases

Non-relational databases designed to handle unstructured, semi-structured, and structured data

- non-relational (NoSQL) databases are non-tabular databases which store data using the data access connectors (such as JSON data formats).
- These databases are built to be flexible, scalable, and capable of rapidly responding to the data management demands of modern businesses. The following are some most-popular types of NoSQL database:
 - flexibility means ability to handle data of different format such as json
 - scalability means the capacity to store the growing datasets
- **Document databases:** The type of databases which store data in the form of documents such as JSON and XML documents. (e.g MongoDB)
- **Key-value stores:** The type of databases which store data by associated group using a unique key value stores. (e.g Redis)
- **Wide-column databases:** The type of database which spread data in a column like format. (e.g Cassandra)
- **Graph databases:** The type of databases which use graph structures to define the relationships between stored data points. Graph databases are useful for identifying patterns in unstructured and semi-structured information. (e.g Neo4j)

Document Database Storage

JSON format

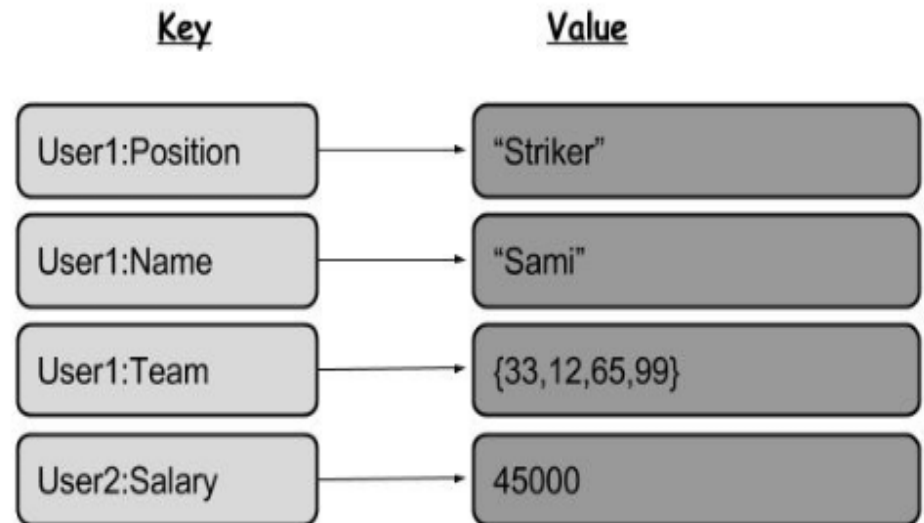
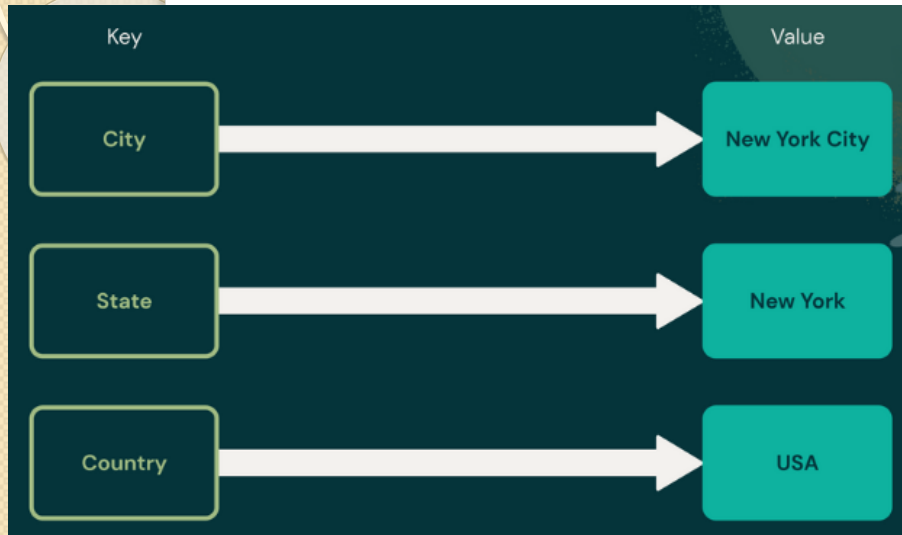
```
{  
  "name": "Dr. Nick",  
  "address": "64 Zoo Street",  
  "officeName": "Victoria Mill Doctors",  
  "patients": [  
    "Joanna Smith",  
    "Joe Bloggs",  
    "Stan Jones"  
  ]  
}
```

XML format

```
<student id="1">  
  <firstName>Greg</firstName>  
  <lastName>Dean</lastName>  
  <certificate>True</certificate>  
  <scores>  
    <module1>70</module1>  
    <module12>80</module12>  
    <module3>90</module3>  
  </scores>  
</student>  
<student ind="2">  
  <firstName>Wirt</firstName>  
  <lastName>Wood</lastName>  
  <certificate>True</certificate>  
  <scores>  
    <module1>80</module1>  
    <module12>80.2</module12>  
    <module3>80</module3>  
  </scores>  
</student>  
</studentsList>
```

Key-value store Database

Redis data examples



Wide-column Database

Column database

AuthorProfile

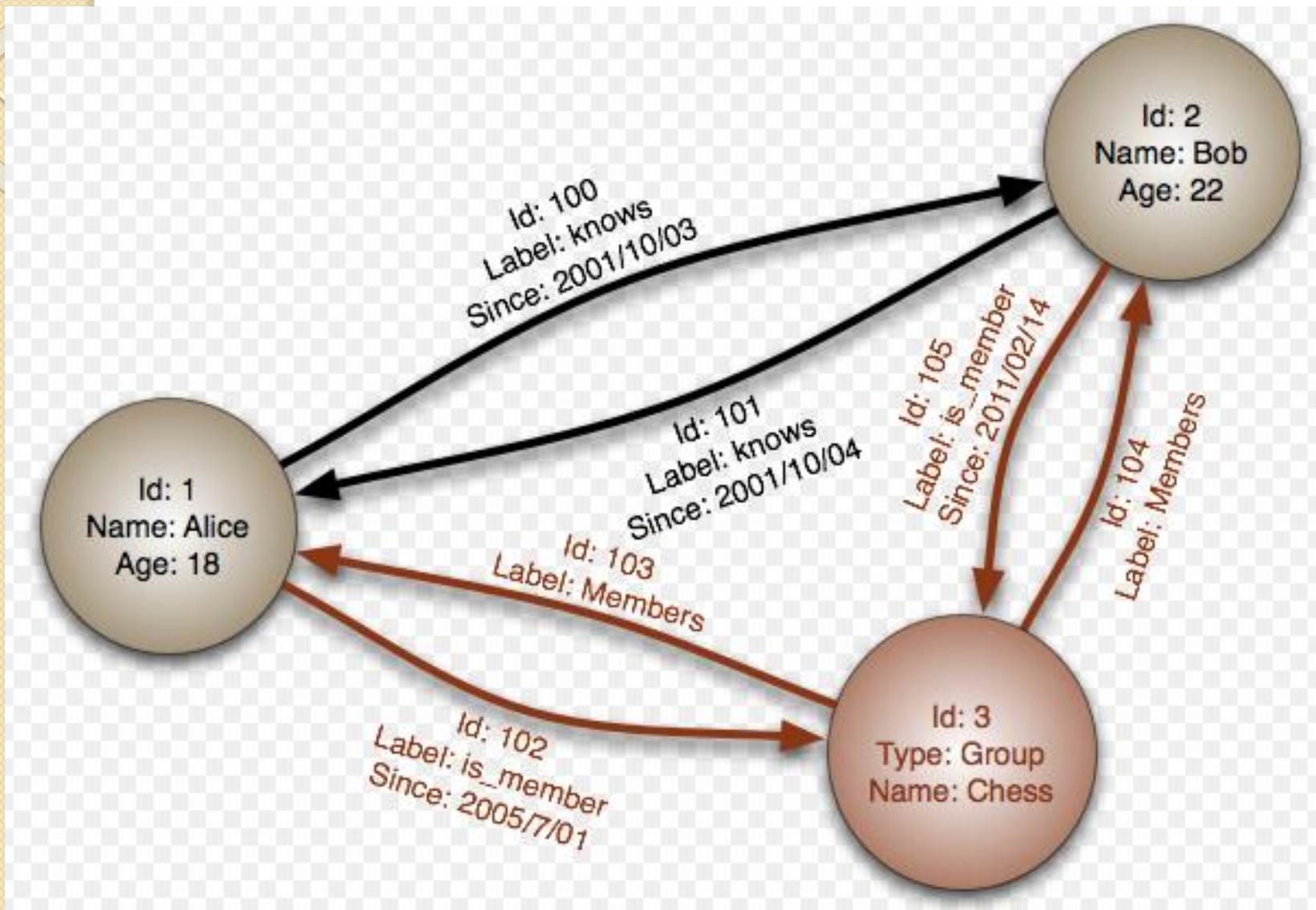
Mahesh	Gender	Expertise	Rank
	Male	ADO.NET, C#, GDI+	102
	03182019	03182019	03182019

David	Gender	Book
	Male	AWS Developer's Guide
	03202019	03202019

Allen	City	Book	Rank
	London	Azure Quickstart	89
	04201019	04201019	04201019

Graph Database

Graph database example



ACID Database

ACID properties in DBMS

➤ DBMS is the management of data that should remain integrated when any changes are done in it. There are four properties described in the database management system, which are known as the ACID properties

- Atomicity: All changes to data are performed as if they are a single operation. That is, the changes should be either completed successfully or failed, no intermediate state
- Consistency: Data is in a consistent state when a transaction starts and when it ends. For example, in an application that transfers funds from one account to another, the total value of funds in both the accounts should be the same at the start and end of each transaction.
- Isolation: The intermediate state of a transaction is invisible to other transactions. It ensures that the first transaction completed before another transaction begins.
- Durability: After a transaction successfully completes, changes to data persist and are not undone, even in the event of a system failure.